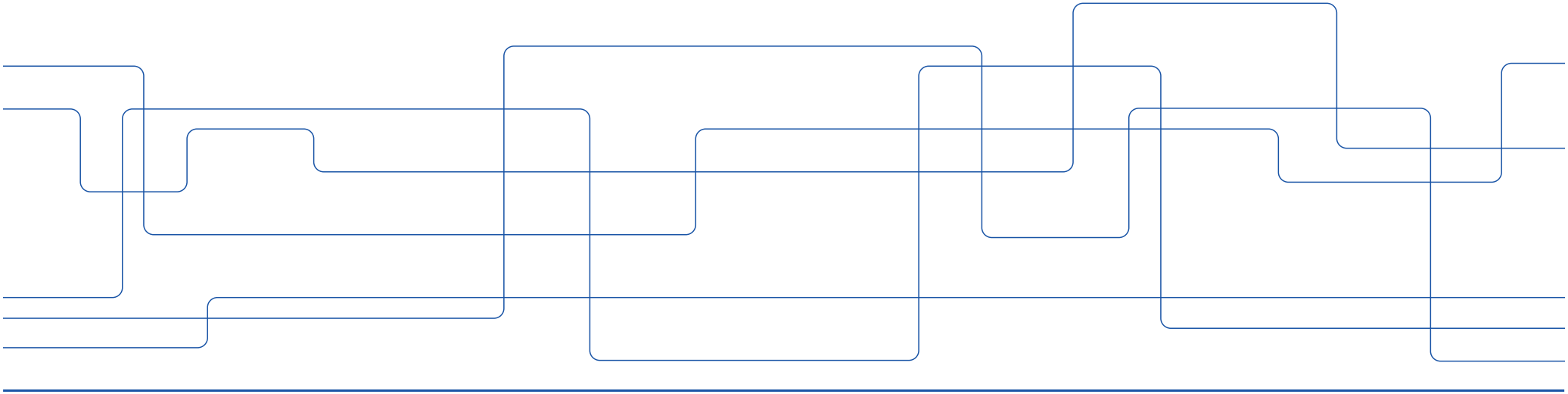


HI1024

Grundkurs i programmering

F4: Iteration & Hopp!



Innehåll!

King 6

Repetition föreläsning #3

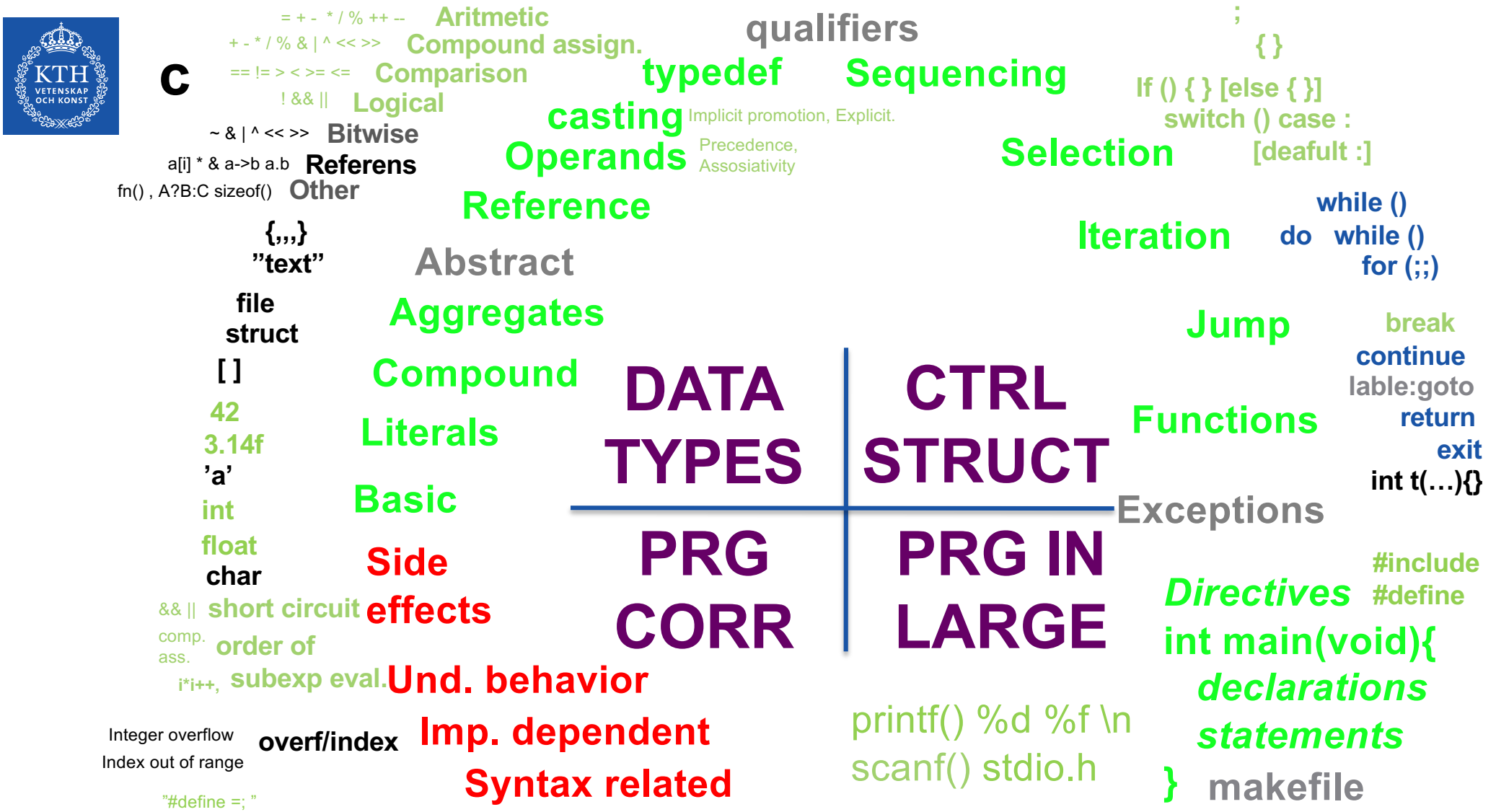
Iteration

Nästlade iterationer

Hopp

Övning




C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
Side effects
Und. behavior
Imp. dependent
Syntax related
Integer overflow
Index out of range
overf/index
”#define =; ”
printf() %d %f \n
scanf() stdio.h
int/int
/*comment*/ or //

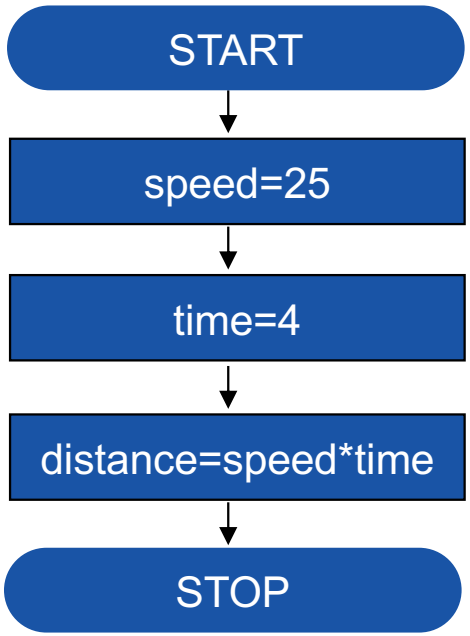
Sekvens & Sammansatt sats…

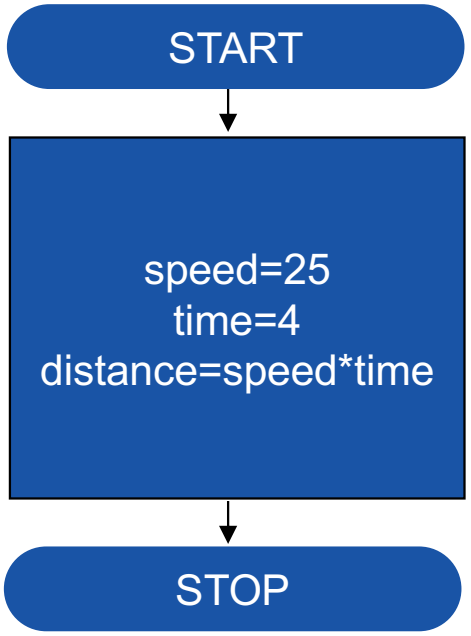
```
speed    = 25;
time     = 4;
distance = speed * time;
```

```
{
  speed    = 25;
  time     = 4;
  distance = speed * time;
}
```

Inget ';'

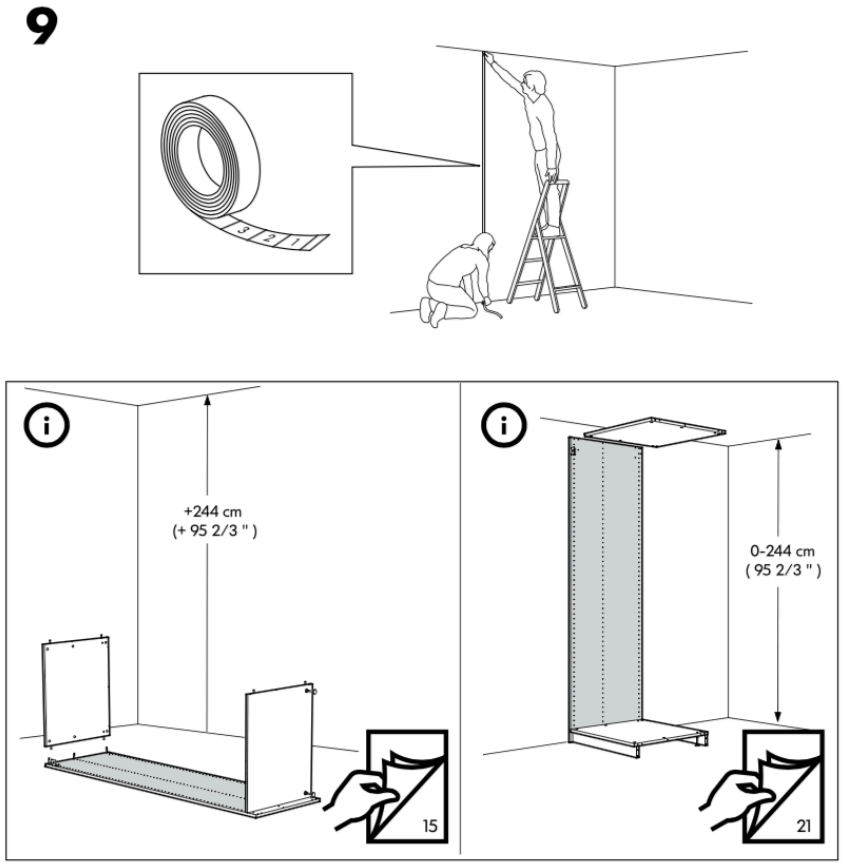
Flowchart

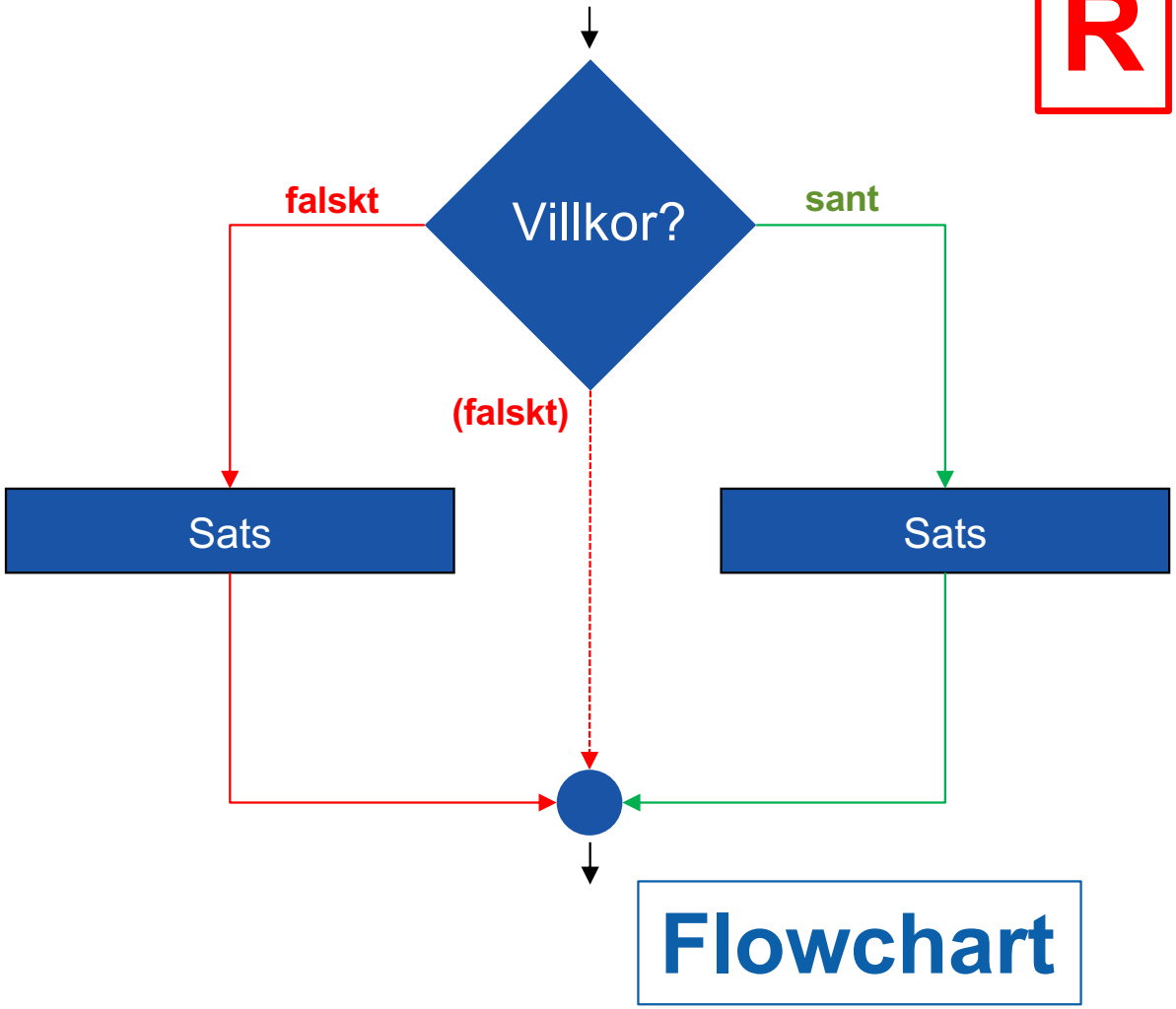




När du skriver t.ex. en tekniska rapporter kan du behöva förklara en algoritm. Sekvens beskrivs så här!

Selektion, if





Det behöver inte finnas någon falskt del, är villkoret falskt hoppar programmet bara över sant delen!

Selektion, if



Parenteser

Måste vara en (1) sats!

```
if ( expression ) statement#1 else statement#2
```

Ett uttryck som när det beräknas bli ett heltal, om det inte är 0 (S) utförs statement#1, om 0 (F) statement#2!

```
if (legs<=0) {
    printf("Ghost?!\n");
} else {
    if (legs<=4) printf("Mammal!\n");
    if ((legs>4) && (legs<8)) printf("Insect!\n");
    if (legs==8) printf("Spider!\n");
    if (legs>8) printf("Creepy!\n");
}
```

Det är helt ok, till och med smart, att göra en sammansatt sats av enbart en (1) sats – det är lätt att glömma bort { } om du/någon i framtiden vill lägga till en sats!

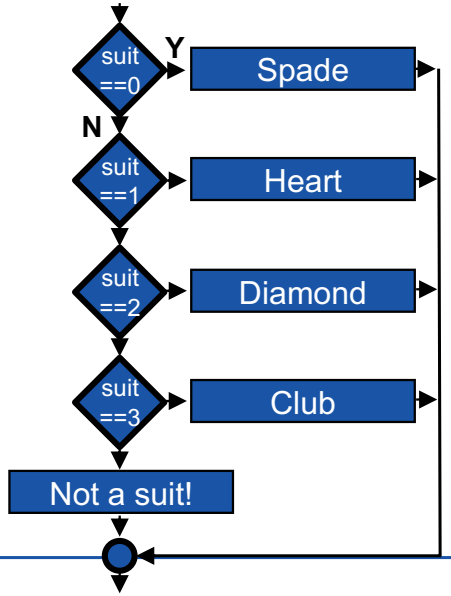
Selection, switch

```
switch ( expression ) {
  case constant-expression : statements
                  …
  case constant-expression : statements
  default : statements /* kan utelämnas! */
}
```

expression **måste** bli ett **heltal**!
constant-expression **måste** vara en **konstant**!
statements behöver **inte** inneslutas i { } **men**
exekveringen fortsätter, även in i nästa case,
till det finns en **break-sats**;

```
switch (theSuit) {
    case  0 : printf("Spade   "); break;
    case  1 : printf("Heart   "); break;
    case  2 : printf("Diamond "); break;
    case  3 : printf("Club    "); break;
    default : printf("Not a suit!");
}
```

Flowchart



Övning #3: Reflektioner…

```c
#include <stdio.h>

int main(int argc, char **argv) {
    int legs;

    printf("My object analyzer, take 2!\n");

    printf("Enter nb of legs : ");
    scanf("%d", &legs);

    if (legs<=4)
        printf("Mammal!\n");
    if ((legs>4) && (legs<8))
        printf("Insect!\n");
    if (legs==8)
        printf("Spider!\n");
    if (legs>8)
        printf("Creepy!\n");

    printf("My analyzer done!\n");
    return 0;
}
```

BRA!

Best practice
aka
Design pattern

```c
#include <stdio.h>

int main(int argc, char **argv) {
    int legs;

    printf("My object analyzer, take 3!\n");

    printf("Enter nb of legs : ");
    scanf("%d", &legs);

    if (legs<=4)
        printf("Mammal!\n");
    else
        if (legs<8)
            printf("Insect!\n");
        else
            if (legs==8)
                printf("Spider!\n");
            else
                printf("Creepy!\n");

    printf("My analyzer done!\n");
    return 0;
}
```

BÄTTRE!

```c
#include <stdio.h>

int main(int argc, char **argv) {
    int legs;

    printf("My object analyzer, take 4!\n");

    printf("Enter nb of legs : ");
    scanf("%d", &legs);

    switch (legs) {
        case 1: case 2: case 3: case 4:
            printf("Mammal!\n"); break;
        case 5: case 6: case 7:
            printf("Insect!\n"); break;
        case 8:
            printf("Spider!\n"); break;
        default:
            printf("Creepy!\n");
    }

    printf("My analyzer done!\n");
    return 0;
}
```

BÄST!

”Alla” seriösa företag har riktlinjer för hur företagets programmerar ska koda, både stilmässigt och principiellt!

Övning #3: Reflektioner…

```c
#include <stdio.h>

int main(int argc, char **argv) {
    int legs;

    printf("My object analyzer, take 3!\n");

    printf("Enter nb of legs : ");
    scanf("%d", &legs);

    if (legs<=4)
        printf("Mammal!\n");
    else
        if (legs<8)
            printf("Insect!\n");
        else
            if (legs==8)
                printf("Spider!\n");
            else
                printf("Creepy!\n");

    printf("My analyzer done!\n");
    return 0;
}
```

Best practice
aka
Design pattern

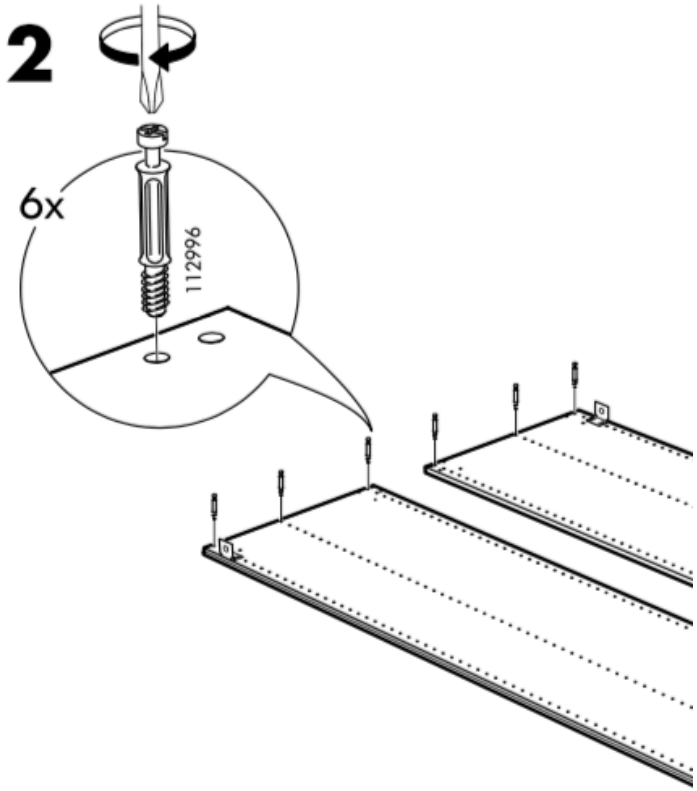
Tomma rader!

Hjälputskrifter!

Indentering!

Plasering av { }

Nytt, Iteration…





Instruktion Fibonacci

$a_1 = 1$

$a_2 = 1$

Upprepa…

$a_n = a_{n-1} + a_{n-2}$

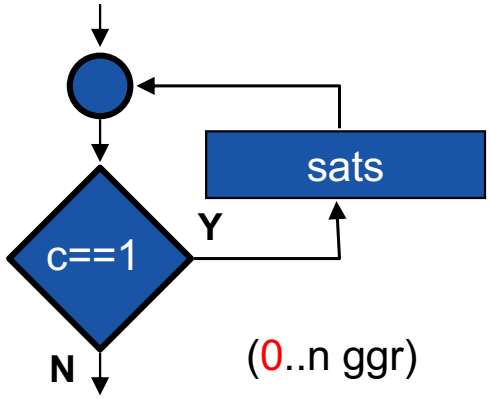
…98 gånger

98 ggr

99 gången

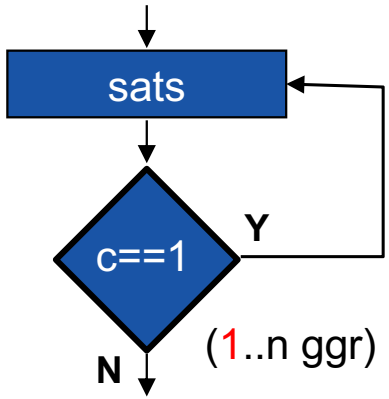
Nytt, Iteration…

Så länge c är 1
utför sats!

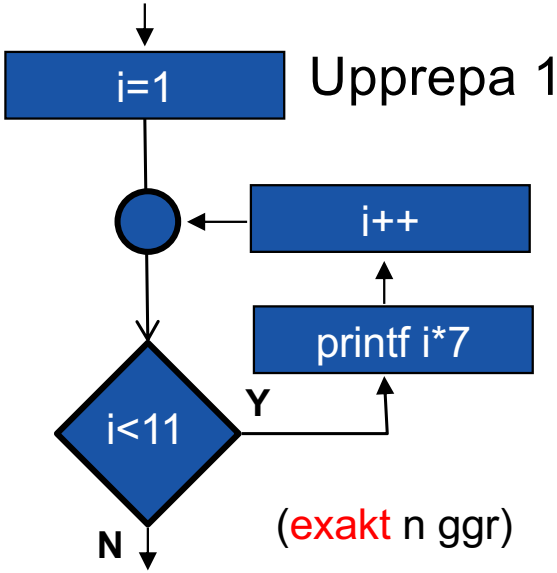


(0..n ggr)

utför sats,
Så länge c är 1!



(1..n ggr)

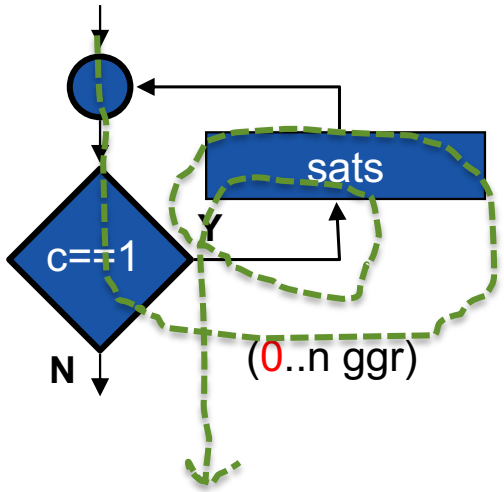


(exakt n ggr)

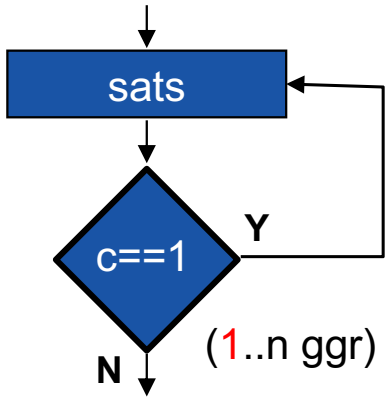
…kan principiellt sett ske på 3 sätt, och c stödjer alla 3 sätten!

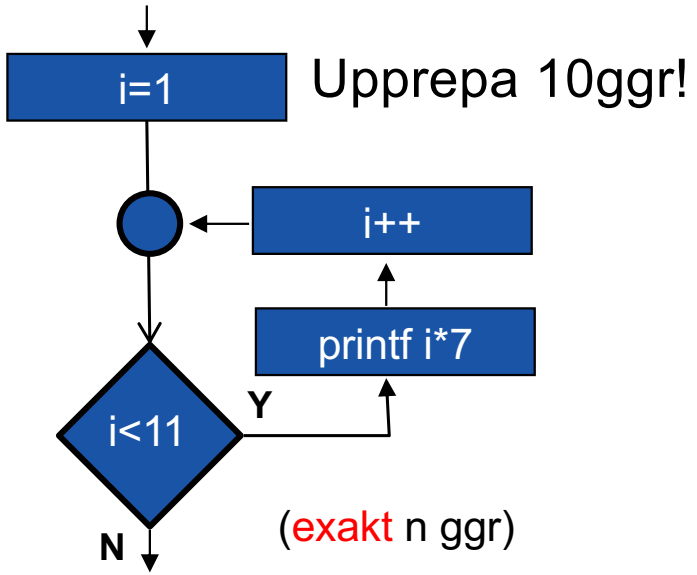
Nytt, Iteration…

Så länge c är 1
utför sats!



utför sats,
Så länge c är 1!





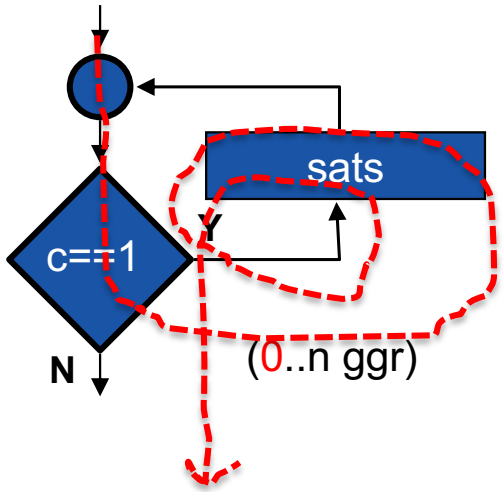
while (expression)
statement

När du inte vet hur många ”varv” som krävs, och det kan hända att det inte ska vara någon gång!

Som med if får statement bara vara en sats, i annat fall måste den vara sammansatt!

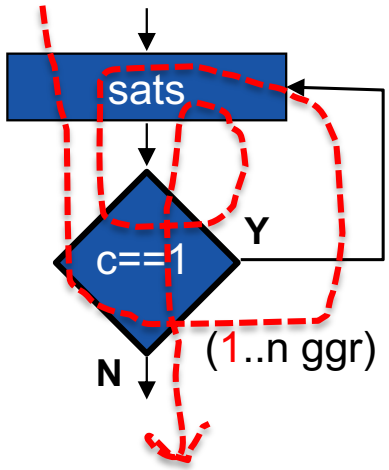
Nytt, Iteration…

Så länge c är 1
utför sats!

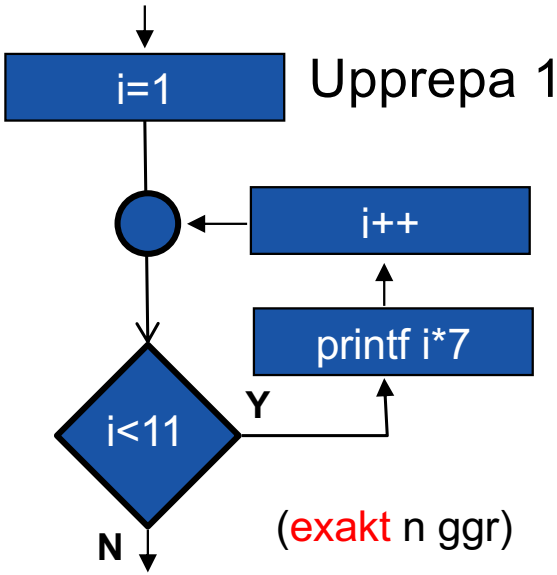


while (expression)
statement

utför sats,
Så länge c är 1!



do
statement
while (expression);



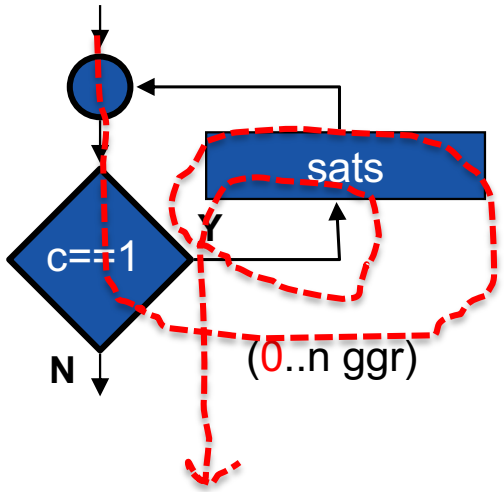
När du inte vet hur många ”varv” som krävs, men det är säkert minst en gång!

Som med if får statement bara vara en sats, i annat fall måste den vara sammansatt!

Nytt, Iteration…

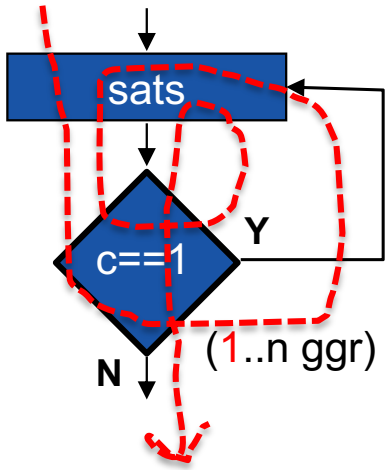
När du vet hur många ”varv” som krävs!

Så länge c är 1
utför sats!

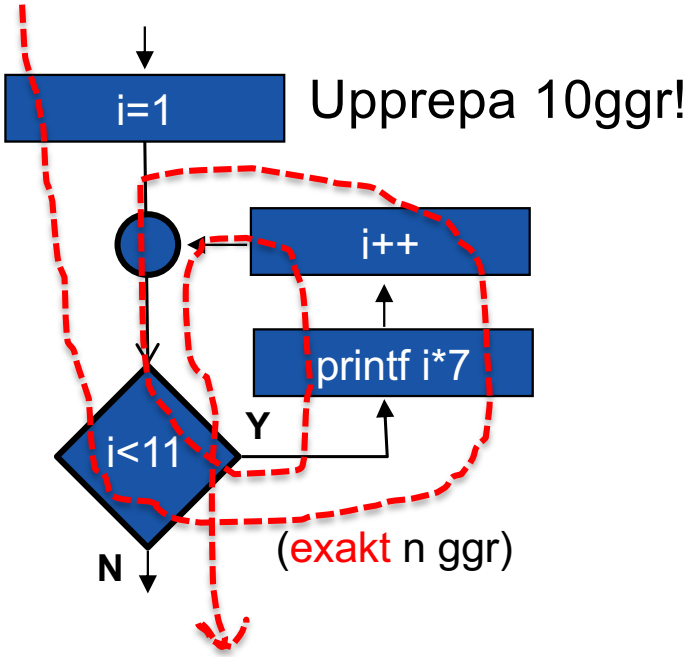


while (expression)
statement

utför sats,
Så länge c är 1!



do
statement
while (expression);



initiering test modifiering
for (expr1 ; expr2 ; expr3)
statement

Som med if får statement bara vara en sats, i annat fall måste den vara sammansatt!

Typexempel while…

while (expression)
statement

- Beräkna roten ur x med den Babylonska iterationsmetoden! (Inte svårare än att gissa ett tal=r_n, beräkna r_{n+1}=medelvärdet(r_n,x/r_n), tills diff(r_n, r_{n+1}) är liten…)

```
// double x, rp=x, rn=x/2, e=maxfel

// Välkomsthälsning

// så länge abs(rn-rp)>e
//.   Debugutskrift
//.   rp=rn
//.   rn=(rn+x/rn)/2

// Visa resultatet
```


Vi vet inte hur många ”varv” som krävs, kanske inget, dock inte så troligt i detta exempel.

Typexempel while…

while (expression)
statement

- Beräkna roten ur x med den Babylonska iterationsmetoden! (Inte svårare än att gissa ett tal=r_n, beräkna r_{n+1}=medelvärdet(r_n,x/r_n), tills diff(r_n, r_{n+1}) är liten…)

```
// double x, rp=x, rn=x/2, e=maxfel

// Välkomsthälsning

// så länge abs(rn-rp)>e
//.   Debugutskrift
//.   rp=rn
//.   rn=(rn+x/rn)/2

// Visa resultatet
```

```c
#include <stdio.h>
#include <math.h>
int main(int argc, char **argv) {
    double x=16.0f, rp=x, rn=x/2, e=0.001;

    printf("Babylon square method\n");

    while (fabs(rn-rp)>e) {
        printf("DBG:%f\n",rn);
        rp=rn;
        rn=(rn + x/rn)/2;
    }

    printf("Square of %f is %.3f\n", x, rn);
    return 0;
}
```

```
Babylon square method
DBG:8.000000
DBG:5.000000
DBG:4.100000
DBG:4.001220
DBG:4.000000
Square of 16.000000 is 4.000
```

Vi vet inte hur många ”varv” som krävs, kanske inget, dock inte så troligt i detta exempel.



Övning while…

while (expression)
statement

Läs in ett tal och skriv ut hur många siffror det har!

(t.ex 1962 har 4 siffror.)

1 **Skriv ner proble-met så att ett barn förstår det!**

Tips: Hur många ggr kan du dela talet med 10 innan det är <10?



Övning while…

while (expression)
statement

Läs in ett tal och skriv ut hur många siffror det har!

(t.ex 1962 har 4 siffror.)

1 **Skriv ner problemet så att ett barn förstår det!**

```
// heltal talet
// heltal siffror=1

// Hälsningstext

// Fråga efter ett tal?
// läs in talet

// Så länge talet>10
//     dela talet med 10
//     siffror+1

// Visa antalet siffror!

// Avslutningstext
```

2 **Gör ett minimalt main() av dina anteckningar!**

```
/* Iterators 0.2 AC */
#include <stdio.h>

int main(void){
    // heltal talet
    // heltal siffror=1
    printf("Iteration Tester (IT)!\n");

    // Fråga efter ett tal?
    // läs in talet

    // Så länge talet>10
    //     dela talet med 10
    //     siffror+1

    // Visa antalet siffror!

    printf("IT Done.\n");
    return 0;
}
```

3 **a..n** **Konstanter**
Variabler
Uttryck &
resultat!

Tips: Hur många ggr kan du dela talet med 10 innan det är <10?

Övning while…

while (expression)
statement

Läs in ett tal och skriv ut hur många siffror det har! (t.ex 1962 har 4 siffror.)

1 **Skriv ner problemet så att ett barn förstår det!**

```
// heltal talet
// heltal siffror=1

// Hälsningstext

// Fråga efter ett tal?
// läs in talet

// Så länge talet>10
//     dela talet med 10
//     siffror+1

// Visa antalet siffror!

// Avslutningstext
```

2

```
/* Iterators 0.2 AC */
#include <stdio.h>

int main(void){
    // heltal talet
    // heltal siffror=1
    printf("Iteration Tester

    // Fråga efter ett tal?
    // läs in talet

    // Så länge talet>10
    //     dela talet med 10
    //     siffror+1

    // Visa antalet siffror!

    printf("IT Done.\n");
    return 0;
}
```

Gör ett r
main() a
anteckn

3

```
/* Iterators 0.2 AC */
#include <stdio.h>

int main(void){
    int n=1962;
    int i=1;

    printf("Iteration Tester (IT)!\n");

    // Fråga efter ett tal?
    // läs in talet

    while (n>10) {
a..n    n=n/10;
        i++;
    }

    printf("Talet bestod av %d siffror!\n",i);

    printf("IT Done.\n");
    return 0;
}
```

Konstanter
Variabler
Uttryck &
resultat!

4

Läs in riktiga värden!

Tips: Hur många ggr kan du dela talet med 10 innan det är <10?

Övning while…

while (expression)
statement

Läs in ett tal och skriv ut hur många siffror det har! (t.ex 1962 har 4 siffror.)

1 **Skriv ner proble-met så att ett barn förstår det!**

```
// heltal talet
// heltal siffror=1

// Hälsningstext

// Fråga efter ett tal?
// läs in talet

// Så länge talet>10
//     dela talet med 10
//     siffror+1

// Visa antalet siffror!

// Avslutningstext
```

2

```
/* Iterators 0.2 AC */
#include <stdio.h>

int main(void){
    // heltal talet
    // heltal siffror=1
    printf("Iteration Tester

    // Fråga efter ett tal?
    // läs in talet

    // Så länge talet>10
    //     dela talet med 10
    //     siffror+1

    // Visa antalet siffror!

    printf("IT Done.\n");
    return 0;
}
```

Gör ett r
main() a
anteckn

3

```
/* Iterators 0.2 AC */
#include <stdio.h>

int main(void){
    int n=1962;
    int i=1;

    printf("Iteration Tester

    // Fråga efter ett tal?
    // läs in talet

    while (n>10) {
        n=n/10;
        i++;
    }

    printf("Talet bestod

    printf("IT Done.\n");
    return 0;
}
```

Konstant
Variabler
Uttryck &
resultat!

a..n

4

```
/* Iterators 0.2 AC */
#include <stdio.h>

int main(void){
    int n;
    int i=1;

    printf("Iteration Tester (IT)!\n");

    printf("Ange ett tal :");
    scanf("%d",&n);

    while (n>10) {
        n=n/10;
        i++;
    }

    printf("Talet bestod av %d siffror!\n",i);

    printf("IT Done.\n");
    return 0;
}
```

```
Iteration Tester (IT)!
Ange ett tal :1962
Talet bestod av 4 siffror!
IT Done.
```

Läs in riktiga värden!

Tips: Hur många ggr kan du dela talet med 10 innan det är <10?



Typexempel do…

```
do
 statement
while ( expression );
```

Läs in tal, avsluta med 0, och skriv ut talens summa!

```
// heltal n
// heltal s=0

// Hälsningstext

// repetera…
//    Fråga efter ett tal?
//    Läs in talet

//    Addera till s
//    …så länge talet är !0

// Visa s!

// Avslutningstext
```

Do-while är väldigt bra vid all typ av användardialog. De kan vara så att flera tal ska matas in, samt att användaren borde få chansen att korrigera sig om något blivit fel

Vi vet inte hur många ”varv” som krävs, men det är helt säkert minst en gång!

Typexempel do…

```
do
 statement
while ( expression );
```

Läs in tal, avsluta med 0, och skriv ut talens summa!

```
// heltal n
// heltal s=0

// Hälsningstext

// repetera…
//    Fråga efter ett tal?
//    Läs in talet

//    Addera till s
//    …så länge talet är !0

// Visa s!

// Avslutningstext
```

```c
/* Iterators 0.3 AC */
#include <stdio.h>

int main(void){
    int n;          /* number to add */
    int s=0;        /* calcylated sum */

    printf("Iteration Tester (IT)!\n");

    do {
        printf("Ange ett tal, avsluta med 0 :");
        scanf("%d",&n);

        s=s+n;      /* ok to add exit zero */
    } while (n);    /* True is everyting except 0! */

    printf("Summan blev : %d\n",s);

    printf("IT Done.\n");
    return 0;
}
```

```
Iteration Tester (IT)!
Ange ett tal, avsluta med 0 :1
Ange ett tal, avsluta med 0 :2
Ange ett tal, avsluta med 0 :3
Ange ett tal, avsluta med 0 :4
Ange ett tal, avsluta med 0 :0
Summan blev : 10
IT Done.
```


Vi vet inte hur många ”varv” som krävs, men det är helt säkert minst en gång!



Övning do: Gissa talet!

Slumptalet är en konstant!

1 **Skriv ner proble-met så att ett barn förstår det!**

```
do
 statement
while ( expression );
```

```
Iteration Tester (IT)!
Guess a number :5
! To Low  !
Guess a number :7
! To High !
Guess a number :6
! CORRECT !
IT Done.
```


Vi vet inte hur många ”varv” som krävs, men det är helt säkert minst en gång!

Övning do: Gissa talet!

Slumptalet är en konstant!

```
do
 statement
while ( expression );
```

1 **Skriv ner proble-met så att ett barn förstår det!**

```
// konstant X 6

// heltal n

// Hälsningstext

// repetera…
//    Fråga efter ett tal?
//    Läs in talet

//    om n=X
//        Rätt, klart!
//    i annat fall
//        om n>X
//            För Högt!
//        i annat fall
//            För Lågt!

//    …så länge n!=X

// Avslutningstext
```

2 **Gör ett minimalt main() av dina anteckningar!**

3 **a..n** **Konstanter Variabler Uttryck & resultat!**

4

```
Iteration Tester (IT)!
Guess a number :5
! To Low  !
Guess a number :7
! To High !
Guess a number :6
! CORRECT !
IT Done.
```

Läs in riktiga värden!

Vi vet inte hur många ”varv” som krävs, men det är helt säkert minst en gång!

Övning do: Gissa talet!

Slumptalet är en konstant!

do
 statement
while (expression);

1 **Skriv ner proble-met så att ett barn förstår det!**

```
// konstant X 6

// heltal n

// Hälsningstext

// repetera…
//    Fråga efter ett tal?
//    Läs in talet

//    om n=X
//        Rätt, klart!
//    i annat fall
//        om n>X
//            För Högt!
//        i annat fall
//            För Lågt!

//    …så länge n!=X

// Avslutningstext
```

2 **Gör ett mi… main() av… anteckni…**

3 **a…**

4

```
/* Iterators 0.4 AC */
#include <stdio.h>
#define X 6

int main(void){
    int n;          /* Guess */

    printf("Iteration Tester (IT)!\n");

    do {
        printf("Guess a number :");
        scanf("%d",&n);

        if (n==X) {
            printf("! CORRECT !\n");
        } else {
            if (n>X)
                printf("! To High !\n");
            else
                printf("! To Low  !\n");
        }
    } while (n!=X);

    printf("IT Done.\n");
    return 0;
}
```

Läs in riktiga värden!

```
Iteration Tester (IT)!
Guess a number :5
! To Low  !
Guess a number :7
! To High !
Guess a number :6
! CORRECT !
IT Done.
```

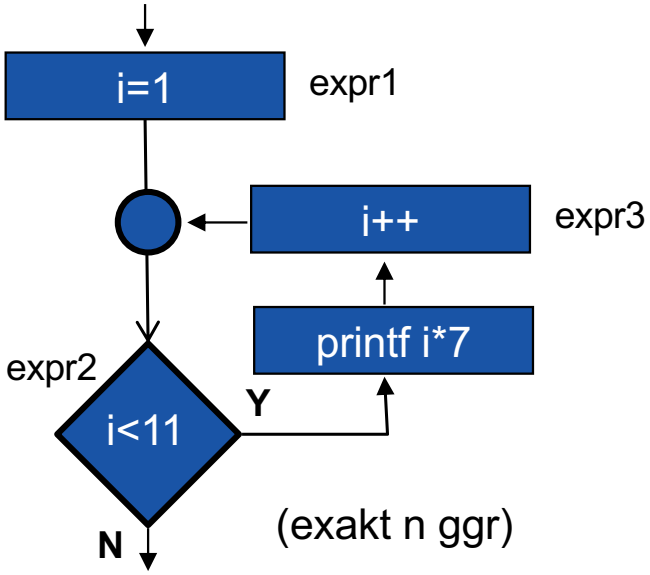

Vi vet inte hur många ”varv” som krävs, men det är helt säkert minst en gång!

Typexempel for…

initiering test modifiering

for (expr1; expr2; expr3)
statement

7:a multiplikations tabell



Lägg specielt märke till:

1. Initieringen sker först och enbart en gång!
2. Initieringen (eller expr2..3) kan utelämnas!
3. Därefter sker testet…
4. Testet kan bli false redan första gången!
5. I annat fall utförs statement
6. ”sist” utförs expr3 innan testet utförs på nytt!

Vi vet hur många ”varv” som krävs!

Typexempel for…

initiering test modifiering

for (expr1; expr2; expr3)

statement

Det är alltid smart att skapa konstanter för slutvillkor #define MAX 11

```c
/* Iterators 0.5 AC */
#include <stdio.h>

int main(void){
    int i;

    printf("Iteration Tester (IT)!\n");

    for (i=1;i<11;i++)
        printf("%3d * 7 =%3d\n",i,i*7);

    printf("IT Done.\n");
    return 0;
}
```

```c
/* Iterators 0.5 AC */
#include <stdio.h>

int main(void){

    printf("Iteration Tester (IT)!\n");

    for (int i=1;i<11;i++)
        printf("%3d * 7 =%3d\n",i,i*7);

    printf("IT Done.\n");
    return 0;
}
```

```
Iteration Tester (IT)!
  1 * 7 =  7
  2 * 7 = 14
  3 * 7 = 21
  4 * 7 = 28
  5 * 7 = 35
  6 * 7 = 42
  7 * 7 = 49
  8 * 7 = 56
  9 * 7 = 63
 10 * 7 = 70
IT Done.
```

Den så kallade loopvariabeln kan deklareras i expr1, vilket är mycket vanligt!

Se upp: den existerar då inte utanför for-satsen!

Vi vet hur många ”varv” som krävs!

Övning for: Medelvärde av 5 tal

initiering test modifiering

for (expr1; expr2; expr3)

statement

1 **Skriv ner proble-met så att ett barn förstår det!**

```
Iteration Tester (IT)!
Enter messaurement #1 : 1
Enter messaurement #2 : 2
Enter messaurement #3 : 3
Enter messaurement #4 : 4
Enter messaurement #5 : 5
The average is :3
IT Done.
```


Vi vet hur många ”varv” som krävs!

Övning for: Medelvärde av 5 tal

initiering test modifiering

for (expr1; expr2; expr3)

statement

```
// konstant C 5
// heltal m, sum=0

// Hälsningstext

// repetera C ggr…
//    Fråga efter ett tal?
//    Läs in talet
//.   Addera till summan

//Skriv ut medelvärdet


// Avslutningstext
```

1 **Skriv ner proble-met så att ett barn förstår det!**

2 **Gör ett minimalt main() av dina anteckningar!**

3 **Konstanter**
Variabler
Uttryck &
a..n resultat!

```
Iteration Tester (IT)!
Enter messaurement #1 : 1
Enter messaurement #2 : 2
Enter messaurement #3 : 3
Enter messaurement #4 : 4
Enter messaurement #5 : 5
The average is :3
IT Done.
```


Vi vet hur många ”varv” som krävs!

Övning for: Medelvärde av 5 tal

initiering test modifiering

for (expr1; expr2; expr3)

statement

// konstant C 5

1 **Skriv ner problemet så att ett barn förstår det!**

// heltal m, sum=0

// Hälsningstext

// repetera C ggr…
// Fråga efter ett tal?
// Läs in talet
//. Addera till summan

//Skriv ut medelvärdet

// Avslutningstext

```c
/* Iterators 0.6 AC */
#include <stdio.h>
#define C 5

int main(void){
    int m, sum=0;

    printf("Iteration Tester (IT)!\n");

    for (int i=0;i<C;i++) {
        printf("Enter messaurement #%1d : ",i+1);
        scanf("%d",&m);
        sum=sum+m;
    }
    printf("The average is :%d\n",sum/C);

    printf("IT Done.\n");
    return 0;
}
```

2

3

4 **Läs in riktiga värden!**

```
Iteration Tester (IT)!
Enter messaurement #1 : 1
Enter messaurement #2 : 2
Enter messaurement #3 : 3
Enter messaurement #4 : 4
Enter messaurement #5 : 5
The average is :3
IT Done.
```

Vi vet hur många ”varv” som krävs!

Nästlade iterationer…

```c
/* Iterators 0.7 AC */
#include <stdio.h>
#define SIZE 4

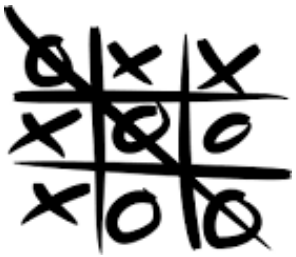
int main(void){

    printf("Iteration Tester (IT)!\n");

    for (int i=0;i<SIZE;i++) {
        for (int j=0;j<SIZE;j++)
            printf("(%d,%d)",i,j);
        printf("\n");
    }

    printf("IT Done.\n");
    return 0;
}
```

```
Iteration Tester (IT)!
(0,0)(0,1)(0,2)(0,3)
(1,0)(1,1)(1,2)(1,3)
(2,0)(2,1)(2,2)(2,3)
(3,0)(3,1)(3,2)(3,3)
IT Done.
```



Tänk på att for-satserna måste ha olika loop-variabler!!!



Bergis?

```c
#include <stdio.h>

int main(void){

for (int n=4; n>0; n--)
  for(int m=0; m<n; m++)
    printf("%d,",m);
}
```


Tänk på att for-satserna måste ha olika loop-variabler!!!

Bergis?

```c
#include <stdio.h>

int main(void){

for (int n=4; n>0; n--)
  for(int m=0; m<n; m++)
    printf("%d,",m);
}
```

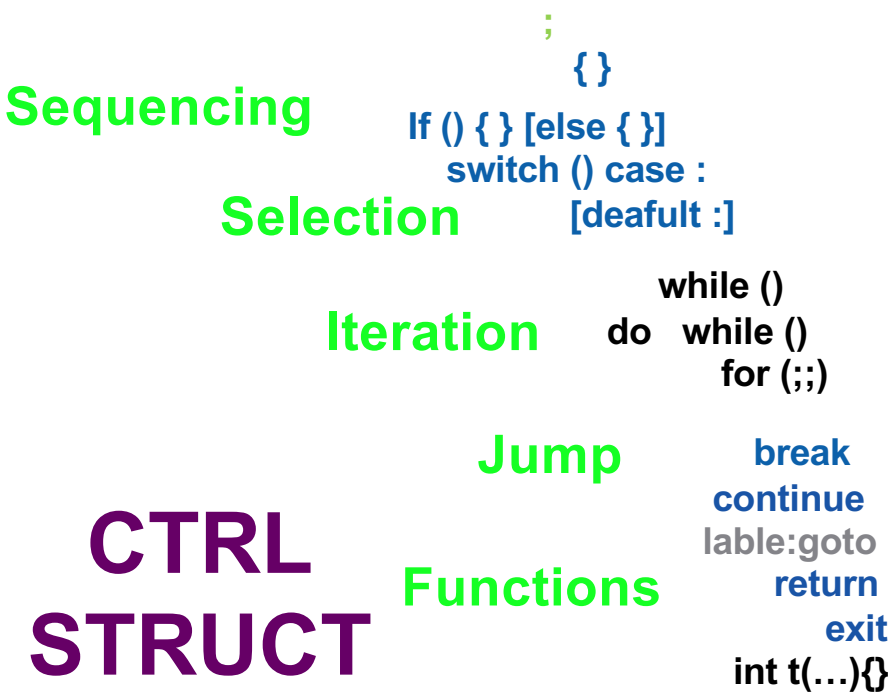


n=4 m=0..3 | n=3 m=0..2 | n=2 m=0..1 | n=1 m=0

0,1,2,3,0,1,2,0,1,0,

Tänk på att for-satserna måste ha olika loop-variabler!!!

Hopp…



Break avslutar en while, do, for-loop eller en switch case-sats. God programmeringssed tillåter dylikt!

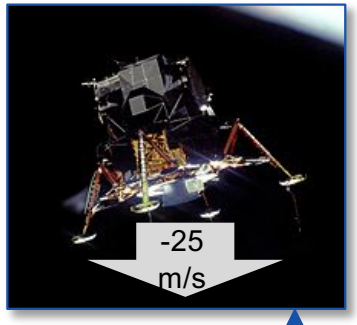
Continue hoppar över återstående satser i en iteration men avslutar inte iterationen. Används sparsamt!

Goto hoppar till en angiven plats i koden. Vi använder **inte** goto i kursen och det används ytterst sällan, möjligen för att hantera sällsynta fel, ute i industrin.

Return avslutar en funktion, och finns det bara en (main) så avslutar det programmet. Måste användas!

Exit avslutar programmet, men vi använder return.

Laboration #1: Månlandning!



Du ska skapa en simulering av en månlandning! Simuleringen börjar 250m ovanför månens yta då månlandaren faller mot månen med hastigheten 25m/s.

Hastighet (v) och höjd (t) en sekund senare är enligt följande formler givet ett gaspådrag (f) om 0-100!

Höjden: $h_{t+1}=h_t+v_t+(0.1f-1.5)/2$

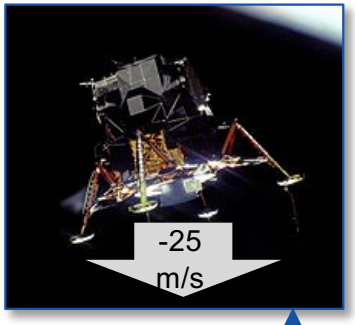
Hastighet: $v_{t+1}=v_t+(0.1f-1.5)$

```
Lunar decent challenge!
You will pilot a lunar decent the last 250m.
Each turn represent 1-second decent time.
Set the throttle for each turn (0-100)
Time Height Velocity Throttle?
  0  250.0    -25.0    0
  1  224.2    -26.5    0
  2  197.0    -28.0    0
  3  168.2    -29.5    0
  4  138.0    -31.0    0
  5  106.2    -32.5    0
  6   73.0    -34.0    0
  7   38.2    -35.5    100
  8    7.0    -27.0    100
FAILED! Crash landing at -18.5 m/s
```

(Siffror i fetstil är inmatade av användaren)

250m

Laboration #1: Månlandning!



Du ska skapa en simulering av en månlandning! Simuleringen börjar 250m ovanför månens yta då månlandaren faller mot månen med hastigheten 25m/s.

Hastighet (v) och höjd (t) en sekund senare är enligt följande formler givet ett gaspådrag (f) om 0-100!

Höjden: $h_{t+1}=h_t+v_t+(0.1f-1.5)/2$

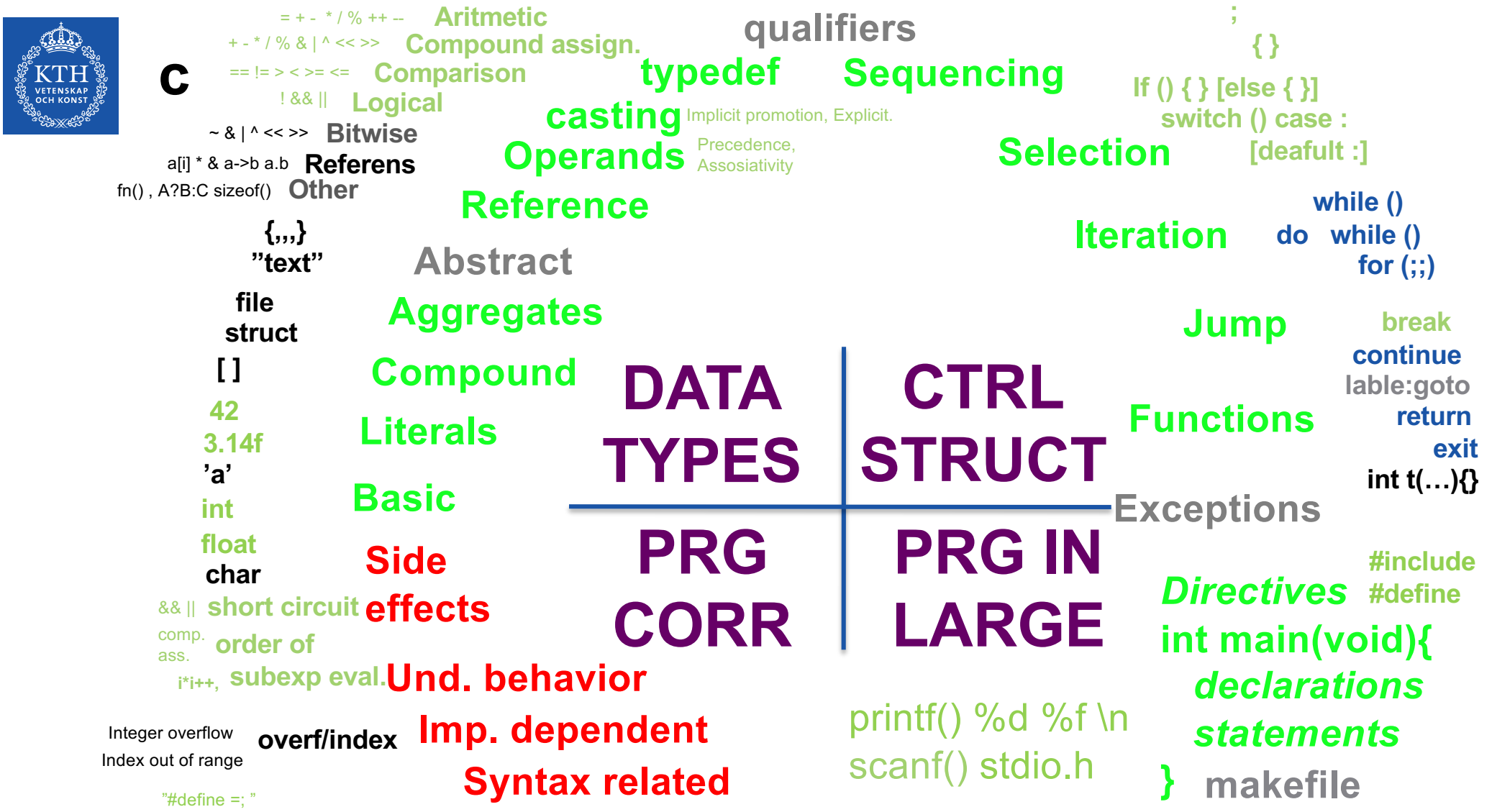
Hastighet: $v_{t+1}=v_t+(0.1f-1.5)$

Skapa variabler med rätt startvärden!
Skriv ut en förklarande text!
Skriv ut tabellrubriken!
Upprepa…
  Skriv ut värdet på relevanta variabler.
  Fråga efter gaspådraget!
  Beräkna variablernas nya värde!
…tills månlandaren ”landat”!
Skriv ut om landningen gick bra eller inte!
Klart!

250m

1 **Skriv ner problemet så att ett barn förstår det!** **2** **3** **4**

Ö5 10/12, Ö6 19/1, Redovisning 22/1!


C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Aggregates
Abstract
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
Side effects
Und. behavior
Imp. dependent
Syntax related
Integer overflow
Index out of range
overf/index
”#define =; ”
printf() %d %f \n
scanf() stdio.h
int/int
/*comment*/ or //

Övningar #3: Kapitel 6:P1,P8 & P11

1 **Skriv ner problemet så att ett barn förstår det!**

Max of
0..n
Numbers

TIPS!
Talen är heltal
Avsluta med 0

One
Month
calendar

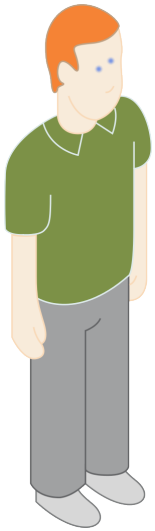
Laboration #1

2 **Gör ett minimalt main() av dina anteckningar!**

3 **Konstanter**
Variabler med
Värde Uttryck
a..n **& resultat!**

4 **Läs in riktiga värden!**

Historik



• 150706 0.1	AC	Skapad.
• 150707 0.2	AC	While.
• 150708 0.5	AC	do, for.
• 150709 0.9	AC	hopp.
• 150710 1.0	AC	Rev A
• 160902 1.1	AC	Mindre uppdateringar
• 190801 2.0	AC	Omarbetad från grunden, samt anpassad till KTH:s nya profil.